ASSIGNMENT 5 – CO5

RELIABLE SOFTWARE WITH RUNTIME POLYMORPHISM AND EXCEPTIONS

DSA0106-C++

Student Name: R. Peeran Basha

Register Number: 192472244

Department: Computer Science and Engineering

Institution: SIMATS Engineering – Saveetha School of Engineering

PART A – DESIGN

TASK A1 – DESIGN THE POLYMORPHIC BILL HIERARCHY

A1.1 – Common Interface

Common data for every bill: patient name or patient ID, bill number and date. Common operations: total() returns the bill amount, type() returns the bill category, and print() displays the bill. total() must be pure virtual because every bill type calculates its amount differently. type() and the destructor are virtual.

A1.2 – Abstract Base Class and Derived Classes

```
class Bill {
protected:
    std::string patient;
    std::string billNo;
public:
    Bill(const std::string& p, const std::string& no)
        : patient(p), billNo(no) {}
    virtual double total() const = 0;
    virtual std::string type() const = 0;
    virtual void print() const;
    virtual ~Bill() = default;
};

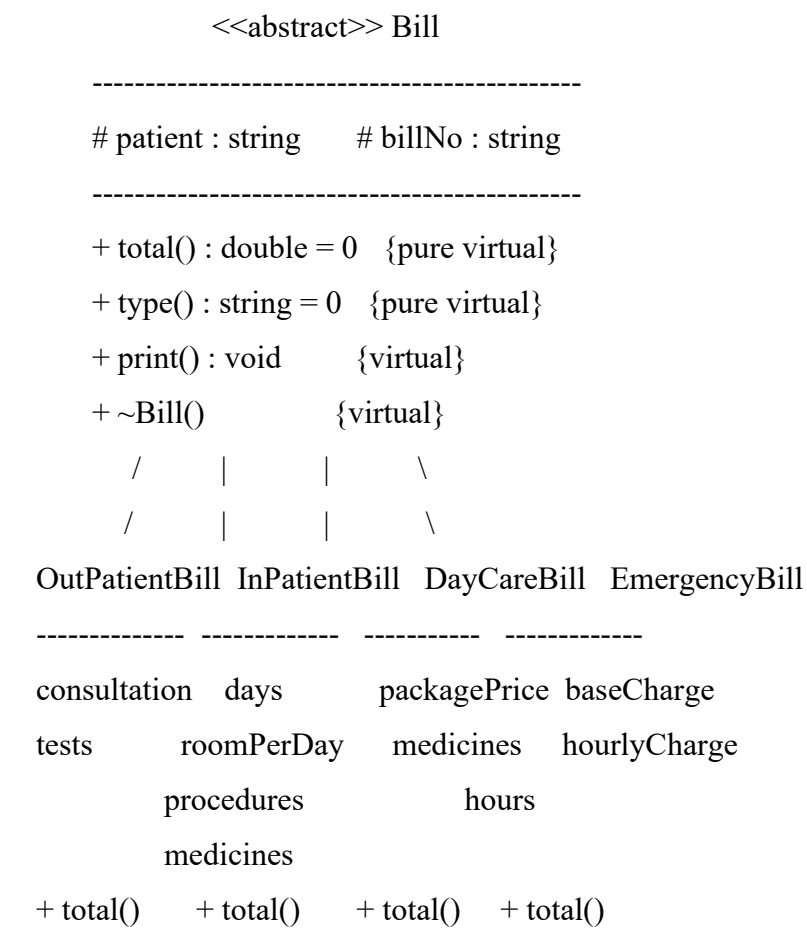
class OutPatientBill : public Bill {
    double consultation, tests;
public:
    double total() const override;
    std::string type() const override;
};

class InPatientBill : public Bill {
    int days;
    double roomPerDay, procedures, medicines;
public:
    double total() const override;
    std::string type() const override;
};
```

```
class DayCareBill : public Bill {
    double packagePrice, medicines;
public:
    double total() const override;
    std::string type() const override;
};

class EmergencyBill : public Bill {
    double baseCharge, hourlyCharge;
    int hours;
public:
    double total() const override;
    std::string type() const override;
};
```

UML-style class diagram:



+ type() + type() + type() + type()

The four derived classes use public inheritance because each one is a kind of Bill. The overridden total() and type() functions provide the correct behaviour for each bill category.

A1.3 – Pseudocode to Process All Bills

```
ALGORITHM Calculate_Day_Total
BEGIN
    SET dayTotal = 0
    FOR each bill pointer in bills array DO
        IF bill pointer is not NULL THEN
            CALL bill pointer -> print()
            SET dayTotal = dayTotal + bill pointer -> total()
        END IF
    END FOR
    DISPLAY "Day total = ", dayTotal
END
```

Alternative Concept and Complexity

A single Bill structure could store a type field and all possible charge fields. total() would use switch(type) to select a formula. Both designs process n bills in O(n) time, with O(1) work per bill. A switch-based structure may store unused fields for every bill. A polymorphic object stores the fields needed by its own type, with implementation-dependent virtual-dispatch overhead. Adding a new type to the switch design requires editing the structure and switch. With inheritance, a new derived class can implement total() and type() without changing the existing classes or processing loop.

TASK A2 – UNDERSTAND RUNTIME BINDING

A2.1 – Early and Late Binding

Early binding selects a function at compile time. A non-virtual function called through Bill* uses the Bill implementation. Late binding selects the overridden function at run time. Because total() is virtual, b->total() calls the total() belonging to the actual object, even when b is declared as Bill*.

A2.2 – Virtual Table Concept

```
Bill vtable:
    total() -> pure virtual / no Bill implementation
    type()  -> pure virtual / no Bill implementation
    ~Bill() -> Bill destructor
```

OutPatientBill vtable:

total() -> OutPatientBill::total()

type() -> OutPatientBill::type()

~Bill() -> OutPatientBill destructor chain

InPatientBill vtable:

total() -> InPatientBill::total()

type() -> InPatientBill::type()

~Bill() -> InPatientBill destructor chain

```
Bill* b = new OutPatientBill(...);
b->total();
```

The object's run-time type selects OutPatientBill::total().

A vtable is a common implementation technique for virtual dispatch; the C++ standard specifies behaviour, not a required physical vtable layout.

A2.3 – Why the Destructor Must Be Virtual

A derived bill is often deleted through a Bill* pointer. If Bill has a virtual destructor, deleting that pointer calls the derived destructor first and then the Bill destructor. Without a virtual base destructor, deleting a derived object through a base pointer has undefined behaviour and may skip derived cleanup.

Predict the Output

The supplied program prints the total first, then destroys the derived object and base object. Predicted output:

500 ~OPBill ~Bill

show() is non-virtual, but it calls the virtual function total(). The call inside show() is still dynamically dispatched, so OPBill::total() returns 500.

If virtual is removed from ~Bill, deleting OPBill through Bill* has undefined behaviour. The C++ language does not guarantee an exact new output. A sanitizer may report a new-delete-type-mismatch or another error depending on the implementation. Record the actual diagnostic from your compiler; do not treat one particular sanitizer message as guaranteed.

Alternative Concept and Complexity

If total() were non-virtual, the program could inspect a type tag and use dynamic_cast or a switch to call the right calculation. A virtual call is O(1) per bill. Trying up to k dynamic_cast operations is

O(k) per bill in the simple model, so processing n bills can be O(nk). A virtual object commonly contains a hidden vptr, but its exact size is implementation-dependent. A switch-based object does not need virtual dispatch metadata but may carry all category fields.

TASK A3 – DESIGN WITH ABSTRACT CLASSES

A3.1 – Payer Interface

```
class Payer {
public:
    virtual double amountCovered(const Bill& bill) const = 0;
    virtual std::string name() const = 0;
    virtual ~Payer() = default;
};

class SelfPay : public Payer {
public:
    double amountCovered(const Bill& bill) const override {
        return bill.total();
    }
    std::string name() const override { return "SelfPay"; }
};

class Insurance : public Payer {
    double coverLimit;
    double coPayPercent;
public:
    Insurance(double limit, double copay)
        : coverLimit(limit), coPayPercent(copay) {}
    double amountCovered(const Bill& bill) const override {
        double eligible = std::min(bill.total(), coverLimit);
        return eligible * (100.0 - coPayPercent) / 100.0;
    }
    std::string name() const override { return "Insurance"; }
};
```

```
class GovernmentScheme : public Payer {
    std::map<std::string, double> packageRates;
public:
    double amountCovered(const Bill& bill) const override;
    std::string name() const override { return "Government"; }
};
```

GovernmentScheme stores package rates in a map. Its amountCovered() looks up the bill type or package code and returns the fixed scheme amount. If the package is not found, it should throw an UnknownPackageError.

A3.2 – Add a New Payer

A CorporateTieUp class can derive from Payer and implement amountCovered() using its corporate discount or maximum cover. The existing SelfPay, Insurance, GovernmentScheme and Bill classes do not need modification. This follows the open/closed principle: open for extension and closed for unnecessary modification.

Alternative Concept and Complexity

A policy can instead be stored as std::function<double(const Bill&)> or a function pointer. Each payer record would hold a callable and any required state, such as a cover limit, could be captured in a lambda or stored in a separate data object. Both virtual dispatch and a callable invocation are O(1) per bill. An abstract Payer object keeps payer data and behaviour together. std::function is flexible but may use extra storage and may allocate for large callables; a function pointer is smaller but cannot directly store state without an extra context object.

TASK A4 – DESIGN THE EXCEPTION STRATEGY

A4.1 – Error Types and Hierarchy

```
std::exception
    |
    +-- BillingError : std::runtime_error
        |
        +-- InvalidChargeError
        +-- ClaimError
        +-- UnknownPackageError
        +-- MissingPatientError

std::bad_alloc (standard allocation failure)
```

std::invalid_argument (invalid numeric text / bad argument)

std::out_of_range (value outside permitted range)

Possible errors: negative consultation or room charges; claim amount above cover; unknown government package; missing patient record; invalid numeric text; and memory allocation failure. The program should use specific exception types so the caller can display a meaningful reason.

A4.2 – Throwing and noexcept

- May throw: constructors that validate input, total() if data is invalid, payer amountCovered(), patient lookup, package lookup and file/database operations.
- Should be noexcept: simple getters that cannot fail, and destructors. A destructor should release resources and must not allow an exception to escape.
- Do not mark a function noexcept if it may call an operation that can throw, unless it catches and handles every exception internally.

Destructors should not throw because they often run during stack unwinding. A second exception escaping while another exception is active can call std::terminate and stop the program.

Alternative Concept and Complexity

Return codes such as enum class Status { Ok, NegativeAmount, ExceedsCover, UnknownPackage } can report errors without exceptions. Normal-path work remains O(1) per claim. Return-code checking adds explicit branches at each call site. Error handling by exceptions has low overhead on the normal path in common implementations, but throwing and catching is comparatively expensive. Return-code results need storage for a status and possibly an error message, while exceptions carry type and message information. Both approaches can be correct; use exceptions for exceptional failures and status values for frequent expected outcomes.

TASK A5 – DESIGN RECOVERY AND RETHROWING

A5.1 – Where to Catch

```
User interface / main:
    catch std::exception:
        show a safe message to the user

Billing service:
    catch ClaimError:
        write audit log
        throw;      // rethrow the same exception
```

Low-level validation:

detect invalid amount or missing data

throw a specific exception

Catch an exception at the level that can make a meaningful decision. A lower layer may log context and use throw; to rethrow the same exception. The top-level application catches std::exception and reports a clear message. A catch(...) at the outermost boundary can prevent an exception from escaping main, but should not hide diagnostic information.

A5.2 – Stack Unwinding and RAII

When an exception leaves a scope, C++ destroys automatic local objects in reverse construction order. This is stack unwinding. A vector, string, file stream or unique_ptr releases its resource in its destructor. Therefore, RAII prevents many leaks even when control exits through an exception.

Predict the Output

Predicted output for the supplied Guard program:

~f ~main caught 42

The local Guard g is destroyed first when f() exits by throwing 42. Then main's Guard m is destroyed as the exception leaves the try block. The matching catch(int) handler prints caught 42. The statement printing not printed is skipped because the throw transfers control to a handler.

Alternative Concept and Complexity

If memory were released manually in every catch block, each error path would need to know which resources had already been allocated. The normal and error operations may still be O(1), but the number of cleanup branches grows as new failure paths are added. Missing one delete or close operation can leak a resource. RAII centralises cleanup in destructors and reduces duplicated cleanup code.

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND VALIDATE POLYMORPHIC BILLING

B1.1 – Complete Implementation

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <iomanip>
#include <stdexcept>
```

```
#include <map>
#include <algorithm>

class Bill {
protected:
    std::string patient;
public:
    explicit Bill(const std::string& p) : patient(p) {}
    virtual ~Bill() = default;
    virtual double total() const = 0;
    virtual std::string type() const { return "Bill"; }
    void print() const {
        std::cout << type() << ' ' << patient << ": " << total() << '\n';
    }
};

class OutPatient : public Bill {
    double consultation, tests;
public:
    OutPatient(const std::string& p, double c, double t)
        : Bill(p), consultation(c), tests(t) {}
    double total() const override { return consultation + tests; }
    std::string type() const override { return "OP"; }
};

class InPatient : public Bill {
    int days;
    double perDay, surgery;
public:
    InPatient(const std::string& p, int d, double rate, double s)
        : Bill(p), days(d), perDay(rate), surgery(s) {}
    double total() const override { return days * perDay + surgery; }
    std::string type() const override { return "IP"; }
```

```
};

class DayCare : public Bill {
    double packagePrice, medicines;
public:
    DayCare(const std::string& p, double package, double meds)
        : Bill(p), packagePrice(package), medicines(meds) {}
    double total() const override { return packagePrice + medicines; }
    std::string type() const override { return "DC"; }
};

class Emergency : public Bill {
    double baseCharge, hourlyCharge;
    int hours;
public:
    Emergency(const std::string& p, double base, double hourly, int h)
        : Bill(p), baseCharge(base), hourlyCharge(hourly), hours(h) {}
    double total() const override { return baseCharge + hourlyCharge * hours; }
    std::string type() const override { return "ER"; }
};

int main() {
    std::vector<std::unique_ptr<Bill>> bills;
    bills.push_back(std::make_unique<OutPatient>("Ravi", 500, 1200));
    bills.push_back(std::make_unique<InPatient>("Meena", 3, 2500, 40000));
    bills.push_back(std::make_unique<DayCare>("Arun", 18000, 2000));
    bills.push_back(std::make_unique<Emergency>("Sara", 3000, 500, 4));

    double sum = 0;
    for (const auto& bill : bills) {
        bill->print();
        sum += bill->total();
    }
```

```
    std::cout << "Total " << sum << '\n';
}
```

Expected output:

```
OP Ravi: 1700
IP Meena: 47500
DC Arun: 20000
ER Sara: 5000
Total 74200
```

B1.2 – Compare with Part A

The implementation follows the abstract Bill design and adds DayCare and Emergency as concrete classes. The vector of unique_ptr<Bill> safely stores different bill types together. The base class destructor is virtual, and total() is pure virtual.

B1.3 – Remove virtual from total()

If total() is non-virtual while it is declared pure virtual, the program is ill-formed: a pure virtual function must be virtual. If the pure specifier is removed as well, a call through Bill* uses the Bill implementation (if one exists), not the derived override. Keep total() virtual for run-time polymorphism.

B1.4 – Remove the Virtual Destructor

Deleting a derived bill through Bill* without a virtual destructor causes undefined behaviour. Compile a separate test with -fsanitize=address and compare it with the corrected version. Record the actual sanitizer result; the exact diagnostic is implementation-dependent.

Alternative Concept and Complexity

With templates, each bill type can be processed in its own container by a generic function. A template call can be statically dispatched without a virtual call. Processing n bills remains O(n). However, one homogeneous template container cannot directly hold arbitrary different types without a variant or another type-erasure technique. Separate containers also make it harder to maintain one mixed chronological bill list.

TASK B2 – ENGINEER ERROR CONDITIONS

Predict the Output

Predicted output for the validate/process program:

```
ok done caught negative [log too large] caught too large
```

100 is valid, so validate prints ok and main prints done. -5 throws invalid_argument; process catches only out_of_range, so it propagates to main, which catches std::exception and prints caught negative.

2e6 throws out_of_range; process logs it and uses throw; to rethrow the same exception. Main catches it and prints caught too large. Only out_of_range is logged because it is the only exception type caught by process.

Complete TODO 1 and TODO 2

```
#include <iostream>
#include <stdexcept>

class ClaimError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};

double approve(double billed, double cover) {
    if (billed < 0 || cover < 0)
        throw std::invalid_argument("negative amount");
    if (billed > cover)
        throw ClaimError("exceeds cover");
    return billed;
}

double settle(double billed, double cover) {
    try {
        return approve(billed, cover);
    } catch (const ClaimError& e) {
        std::cout << "audit: " << e.what() << '\n';
        throw;
    }
}

int main() {
    double cases[][2] = {{40000, 50000}, {60000, 50000}, {-1, 50000}};
    for (auto& c : cases) {
        try {
```

```
            double paid = settle(c[0], c[1]);
            std::cout << "approved " << paid << '\n';
        } catch (const ClaimError& e) {
            std::cout << "claim rejected: " << e.what() << '\n';
        } catch (const std::exception& e) {
            std::cout << "error: " << e.what() << '\n';
        }
    }
}
```

Expected output:

```
approved 40000
audit: exceeds cover
claim rejected: exceeds cover
error: negative amount
```

Replacing throw; with throw e; throws the caught exception object as a new exception expression. This can copy/slice the object to the static type of the catch parameter (ClaimError here) and loses the original exception object's identity. throw; rethrows the original active exception and preserves its dynamic type and identity. In this exact code the catch parameter is ClaimError, so visible message and type may appear the same, but throw; is the correct general rethrow operation.

Error Test Plan

Test input	**Expected handling**	**What it verifies**
Negative charge	invalid_argument	Input validation
Claim exceeds cover	ClaimError	Business rule validation
Unknown package	UnknownPackageError	Package lookup
Text instead of number	invalid_argument from stod	Input parsing
Impossibly large allocation	bad_alloc if allocation fails	Allocation failure path
Top-level catch(...)	Application reports controlled failure	Exception boundary and cleanup

Alternative Concept and Complexity

std::optional<double> can represent approval or rejection without throwing. Each claim is O(1) in either approach. For 1,000,000 claims, both are O(n). With 0% invalid claims, optional/return status

is often efficient because no exception is thrown. At 1% and 50% invalid rates, throwing exceptions can become much slower because constructing and propagating exceptions is expensive. std::optional stores a value plus a presence state, adding a small fixed amount of space per result; an exception object is generally created only on the error path. Actual performance depends on compiler, runtime and hardware and must be measured.

TASK B3 – VIRTUAL FUNCTIONS VS SWITCH VS EXCEPTIONS: THE COST

B3.1 – Correctness on Small Data

Use the same ten bills and the same input values in both implementations. The virtual version calls total() through Bill pointers. The switch version stores a type tag and calls a switch-based calculation. Compare the two totals and assert that their difference is less than a small floating-point tolerance, such as 0.000001.

B3.2 – Benchmark Procedure

Compile optimized builds:

```
g++ -std=c++17 -O2 -DNDEBUG virtual_bench.cpp -o virtual_bench
g++ -std=c++17 -O2 -DNDEBUG switch_bench.cpp -o switch_bench
```

Run each benchmark several times:

```
./virtual_bench 1000
./virtual_bench 1000000
./virtual_bench 10000000

./switch_bench 1000
./switch_bench 1000000
./switch_bench 10000000
```

Use std::chrono::steady_clock, generate the data before starting the timer, run the calculation loop, and print a checksum so the compiler cannot remove the work. Repeat at least five times and report the median. Do not include object creation or random-number generation in the timed section unless the experiment specifically measures those operations.

B3.3 – Exception Cost Procedure

Create one million claims with 0%, 1% and 50% invalid entries. Run the same validation rule once with a return-code result and once with exceptions. Use identical data and timing boundaries. Record the time for each percentage. The normal path for exceptions may be competitive, but exception-heavy cases commonly become significantly slower as the invalid rate increases.

B3.4 – Engineering Trade-Off

Approach	Strength	Limitation / suitable use
Virtual functions	Easy extension and mixed-type collections	Small dispatch overhead; good for extensible bill hierarchies
Switch/type tag	Simple and can be compact	Every new type changes central switch logic
Exceptions	Clear separation of error path from normal path	Expensive when errors are frequent
Return status	Predictable control flow and often efficient for frequent rejection	Callers must check and propagate status

Alternative Concept – std::variant

std::variant<OutPatientBill, InPatientBill, DayCareBill, EmergencyBill> stores one of the listed types in each element. std::visit calls the matching operation. This is a form of closed-set polymorphism: adding a new alternative changes the variant type and visitor. Processing n elements is O(n), and each visit is constant-time in the usual model. The element needs enough storage for the largest alternative plus a discriminator and alignment padding. Measure it with sizeof(Variant).

```
using BillVariant = std::variant<OutPatientBill, InPatientBill,
                                 DayCareBill, EmergencyBill>;

double amount(const BillVariant& bill) {
    return std::visit([](const auto& item) {
        return item.total();
    }, bill);
}
```

Benchmark table to complete after execution:

Bill count	Virtual median (ms)	Switch median (ms)	Variant median (ms)
1,000	_____	_____	Not required / optional
1,000,000	_____	_____	_____
10,000,000	_____	_____	_____

Invalid rate	Return-code time (ms)	Exception time (ms)
0%	_____	_____
1%	_____	_____
50%	_____	_____

TASK B4 – FINAL ENGINEERING DECISION

B4.1 – Add a New Bill and Payer

Example extension: add a PharmacyBill derived from Bill, and a CorporatePayer derived from Payer. PharmacyBill implements total() as medicine charges plus dispensing fee. CorporatePayer implements amountCovered() using a corporate cover limit and negotiated percentage. No existing bill or payer class needs modification; only the new derived classes and object creation/configuration are added.

```
class PharmacyBill : public Bill {
    double medicines, dispensingFee;
public:
    PharmacyBill(const std::string& patient, double meds, double fee)
        : Bill(patient), medicines(meds), dispensingFee(fee) {}
    double total() const override { return medicines + dispensingFee; }
    std::string type() const override { return "PHARMACY"; }
};

class CorporatePayer : public Payer {
    double limit;
    double coveredPercent;
public:
    CorporatePayer(double l, double p) : limit(l), coveredPercent(p) {}
    double amountCovered(const Bill& bill) const override {
        double eligible = std::min(bill.total(), limit);
        return eligible * coveredPercent / 100.0;
    }
    std::string name() const override { return "Corporate"; }
};
```

B4.2 – Resource Safety

Run the complete billing program with AddressSanitizer, including successful claims and thrown/caught exceptions. A clean run should finish without AddressSanitizer memory errors. Attach the actual sanitizer output from your computer. The use of std::vector, std::string, std::map and std::unique_ptr supports automatic resource management.

B4.3 – Production Decision

The polymorphic design is suitable for a hospital group that adds bill and payer types regularly. It provides a common interface, supports mixed bill collections and allows new categories to be added without rewriting the central processing loop. Exceptions are appropriate for exceptional failures such as missing records, invalid configuration or allocation failure. For frequent business rejections, such as routine claims above a cover limit, a return status or expected-like result may be more efficient and explicit. The production design should combine polymorphism for extensibility, RAII for resource safety, and measured error reporting suited to the frequency of each condition.

Alternative Concept – Configuration Plug-ins

Bill and payer rules could be loaded from configuration files instead of adding C++ classes. A rule engine interprets the configured rate, package and eligibility rules. Each bill calculation may require rule lookup and interpretation, which can be more expensive than a direct virtual call, but it allows authorised rate updates without recompiling. Configuration needs validation, versioning, audit trails and strict access control. C++ classes provide stronger compile-time checks but require development and deployment for every new type. A practical hospital platform can use stable C++ interfaces with validated, version-controlled configuration for rates and package values.